

Memory Management

1 Buddy system

1. Consider a buddy system of size 1 MByte. Show the memory state graphically after the following actions, assuming memory is initially empty:

- (a) Request $A = 70$ KByte
- (b) Request $B = 35$ KByte
- (c) Request $C = 80$ KByte
- (d) Return A
- (e) Request $D = 60$ KByte
- (f) Return B
- (g) Return D

Solution:

```
1024K
128K 128K 256K 512K
A
128K 64K 64K 256K 512K
A B
128K 64K 64K 128K 128K 512K
A B C
128K 64K 64K 128K 128K 512K
B C
128K 64K 64K 128K 128K 512K
B D C
128K 64K 64K 128K 128K 512K
D C
256K 128K 128K 512K
C
```

2. In the standard buddy system, processes ultimately get memory blocks of powers of 2. Suppose instead we want to use a Fibonacci sequence.

- Explain how such a system works using a short example (tip: assume the total memory size is a (large) Fibonacci number).

Solution: Consider a buddy system of size 89 KB.

- (a) Request $X = 12$ KByte
- (b) Request $Y = 12$ KByte
- (c) Return X
- (d) Request $Z = 4$ KByte

```
89K
13K 21K 55K
X
13K 8K 13K 55K
X Y
13K 8K 13K 55K
Y
13K 3K 5K 13K 55K
Z Y
```

- Does this system perform better or worse than the standard buddy system in terms of internal fragmentation? Why (or why not)?

Solution: Assume a uniform distribution of request sizes (between some unspecified bounds, but large enough). Let the n -th Fibonacci term be f_n , and recall:

$$\lim_{n \rightarrow \infty} \frac{f_{n+1}}{f_n} = \frac{1 + \sqrt{5}}{2} = \varphi.$$

- **Standard buddy system:** each allocated block is at least half full, and at most completely full. On average: $3/4$ full. So for a requested block, on average we allocate

$$\frac{4}{3} - 1 \approx 33\%$$

extra memory.

- **Fibonacci buddy system:** each allocated block is at least f_n/f_{n+1} full, and at most completely full. On average:

$$\frac{1}{2} \cdot \frac{2}{1 + \sqrt{5}} + \frac{1}{2} \cdot 1 \approx 81\% \text{ full.}$$

So for a requested block, on average we allocate

$$(81\%)^{-1} - 1 \approx 23\%$$

extra memory.

Hints about Fibonacci:

$$\varphi = \frac{1 + \sqrt{5}}{2}, \quad \psi = \frac{1 - \sqrt{5}}{2} = 1 - \varphi,$$

$$f_n = \frac{\varphi^n - \psi^n}{\varphi - \psi} = \frac{\varphi^n - \psi^n}{\sqrt{5}},$$

$$\varphi^2 = \varphi + 1, \quad \varphi^{n+1} = \varphi^n + \varphi^{n-1},$$

$$f_{n+1} = f_n + f_{n-1} \Rightarrow \frac{f_{n+1}}{f_n} = 1 + \frac{f_{n-1}}{f_n} \Rightarrow \varphi = 1 + \frac{1}{\varphi}.$$

Paging

1 Size of the page tables of a process

Assume a 32-bit machine with virtual addresses split into 4 parts of 10, 8, 6 and 8 bits respectively. We use these addresses in a 3-level page table system, where the first 10 bits are used for level 1, the next 8 bits for level 2, etc. A single page table entry is 4 bytes. The system is byte-addressed.

1. What is the page size in this system?

Solution: 2^8 bytes.

2. Suppose a process needs 256K of memory (starting from address 0). How much space do this process's page tables take?

Solution:

- 1 page = 2^8 bytes
- 1 level-3 (L3) table covers 2^6 pages
- 1 level-2 (L2) table covers 2^8 L3 tables
- 256 KB = 2^{18} B, so 2^{10} pages, hence 24 L3 page tables, 1 L2 page table and 1 L1 page table.
- Total:

$$4 \times (24 \times 2^6 + 1 \times 2^8 + 1 \times 2^{10}) = 9216 \text{ B.}$$

3. Suppose a process has a code segment of 48K starting from (virtual) address 0x10000000, a 600K data segment starting from address 0x80000000, and a stack segment of 64K starting from address 0xF0000000. How much space does the page table of this process require?

Solution: $0x10000000 = 2^{28}$ bytes.

- 48KB: 192 pages $\Rightarrow$ 3 L3 tables, 1 L2 table, 1 L1 table.
- 600KB: 2400 pages $\Rightarrow$ 38 L3 tables, 1 L2 table, 1 L1 table.
- 64KB: 256 pages $\Rightarrow$ 4 L3 tables, 1 L2 table, 1 L1 table.
- Check for overlapping tables based on their address coverage:
 - 1 L3 table covers $2^6 \cdot 2^8 = 2^{14}$ B = 0x40000.
 - 1 L2 table covers $2^8 \cdot 2^6 \cdot 2^8 = 2^{22}$ B = 0x4000000.
 - 1 L1 table covers $2^{10} \cdot 2^8 \cdot 2^6 \cdot 2^8 = 2^{32}$ B = 0x100000000.
- In this case, all L1 tables coincide.
- Total:

$$4 \times (45 \times 2^6 + 3 \times 2^8 + 1 \times 2^{10}) = 18688 \text{ B.}$$

2 Determine the number of levels in a multilevel paging system

Assume a machine has 36-bit virtual addresses and pages of size 8K. A single page entry is 8 bytes long.

1. How many pages does the virtual address space contain?

Solution:

$$\frac{2^{36}}{8K} = \frac{2^{36}}{2^{13}} = 2^{23} \text{ pages.}$$

2. Suppose an average process is 8 GByte in size. How many levels are most suitable if we use a multilevel page table (consider 1 to 3 levels)? Weigh the pros and cons. For the 2-level system, choose the size of a level-2 page table so that it fits exactly in one page. For three levels, use a more or less equal split for each field.

Solution:

- **Single level:** L1 + offset = 23 + 13 bits.

$$2^{23} \text{ PTEs} \times 8 \text{ bytes} = 2^{26} \text{ bytes} = 64 \text{ MB.}$$

- **2 levels:** L1 L2 offset = 13 10 13 bits.

- 1 page = 2^{13} bytes, so it contains 2^{10} PTEs (because each PTE is 8 bytes).
- 1 L2 table covers 2^{10} pages.
- 8 GB = 2^{33} B, so 2^{20} pages, hence 2^{10} L2 tables and 1 L1 table.
- Total:

$$8 \times (2^{10} \cdot 2^{10} + 1 \cdot 2^{13}) = 8256 \text{ KB.}$$

- **3 levels:** L1 L2 L3 offset = 7 8 8 13 bits.

- 1 page = 2^{13} bytes.
- 1 L3 table covers 2^8 pages.
- 1 L2 table covers 2^8 L3 tables.
- 8 GB = 2^{33} B, so 2^{20} pages, hence 2^{12} L3 tables, 16 L2 tables and 1 L1 table.
- Total:

$$8 \times (2^{12} \cdot 2^8 + 16 \cdot 2^8 + 1 \cdot 2^7) = 8225 \text{ KB.}$$

- The 1-level system clearly requires much more memory than 2 or 3 levels. Between the latter two, the difference is small, but an extra level adds overhead, so 2 levels seems the most attractive.

3 Count the number of TLB misses

Consider the following code. Assume the binary code fits on one page and the stack also fits on a single page. You may also assume that when calling the function, the stack page and code page are already in the TLB. Assume an integer is 4 bytes and a page contains 4096 bytes.

```
int a[1024][1024], b[1024][1024], c[1024][1024];

multiply() {
    unsigned i,j,k;
    for(i=0; i< 1024; i++)
        for(j=0; j< 1024; j++)
            for(k=0; k< 1024; k++)
                c[i][j] += a[i][k] * b[k][j];
}
```

Given that the TLB has only 8 entries and uses an LRU replacement policy, how many TLB misses occur during execution?

Solution:

- The stack and code pages always remain in the TLB, so 6 TLB entries remain for data.
- Each array consists of 1024 pages:

- Page 0 of a contains $a[0][0]$ to $a[0][1023]$
- Page 1 of a contains $a[1][0]$ to $a[1][1023]$
- ...
- Fix i :
 - Requests: $a[i][0], b[0][0], c[i][0], a[i][1], b[1][0], c[i][0], \dots$
 - a and c stay within one page each $\Rightarrow$ 1 miss for a , 1 miss for c .
 - b uses a different page each time $\Rightarrow 1024 \times 1024$ misses.
 - Total for fixed i : $2 + 1024 \times 1024$.
- Repeat for all i :

$$1024 \times (2 + 1024 \times 1024) = 1,073,743,872.$$

4 Demand paging: good or bad

Which of the following programming techniques works well with a demand-paged system?

1. Stack
2. Hash table
3. Sequential search
4. Binary search
5. Pure code
6. Vector operations (e.g. vector multiplication)
7. Indirect addressing

Solution:

1. Stack: + (good)
2. Hash table: – (bad)
3. Sequential search: +
4. Binary search: – (bad initially, later better)
5. Pure code: + (if there are few “far” function calls)
6. Vector operations: +
7. Indirect addresses/pointers: typically – (but + if pointers are close together)

5 Useful interventions for demand paging

Suppose we collect statistics on a demand-paged system and observe: CPU 20% utilization, paging disk 97.3%, other I/O devices only 5%. Which interventions seem effective?

1. Install a faster CPU
2. Install a larger paging disk
3. Increase the degree of multiprogramming
4. Decrease the degree of multiprogramming
5. Install more memory
6. Install a faster hard disk
7. Allow prepaging
8. Increase the page size

Solution:

1. Faster CPU: little effect
2. Larger paging disk: little effect
3. Increase multiprogramming: typically worse (more processes $\Rightarrow$ more memory needed)
4. Decrease multiprogramming: typically better
5. More memory: better
6. Faster hard disk: small improvement
7. Prepaging: depends on the situation
8. Increase page size: depends on the situation

6 Inverted versus standard page tables

If virtual addresses are 64 bits long, page size is 1 KByte, physical memory size is 64 MByte, and a PTE takes two bytes: what is the total size of the inverted and standard page table (byte-addressed)? First estimate how many bytes an inverted page table entry needs by determining which fields must be present at minimum and how many bits each needs.

Solution:

• Standard page table:

- 64-bit virtual address space: 2^{64} bytes
- Number of virtual pages:

$$\frac{2^{64}}{1K} = \frac{2^{64}}{2^{10}} = 2^{54}.$$

- Size:

$$2^{54} \text{ entries} \times 2 \text{ bytes} = 2^{55} \text{ bytes} = 32768 \text{ TByte.}$$

• **Inverted page table:**

- Number of physical frames:

$$\frac{64M}{1K} = \frac{2^{26}}{2^{10}} = 2^{16} = 65536 \text{ entries.}$$

- Bytes needed per IPT entry (example estimate): 10 bytes (80 bits).
 - * Virtual page number: 2^{54} possibilities $\Rightarrow$ 54 bits
 - * Pointer to next entry: 2^{16} possibilities $\Rightarrow$ 16 bits
 - * Process ID and flags: e.g. 10 bits (arbitrarily chosen)
- Total size:

$$65536 \times 10 = 640 \text{ KByte.}$$

7 Inverted page table

Consider a system with 16KB memory, pages of size 1KB, an inverted page table, and a fully associative TLB with 2 entries using LRU. The hash function is

$$f(\text{PID}, \text{VPN}) = (10 \cdot \text{PID} + \text{VPN}) \bmod 24.$$

The system needs 0.5ns to read the TLB, 100ns to read main memory, and 10ms to access the hard disk.

Fill in the page request table using the given hash anchor table and IPT. In the IPT, “T” indicates there is no next page table entry. In the TLB column, give the TLB contents after translation using (VPN, Frame number) per entry; use - if empty. In the columns “Frame number”, “TLB hit”, and “Page fault”, give respectively the physical frame number, whether there was a TLB hit, and whether it caused a page fault. Finally, in the last column, give the time needed for the translation. Assume the buffer can be updated in parallel after translating/executing the next instruction, so you do not count time for that. In case of a page fault, you do not need to update data structures; only give the time needed to discover the page fault.

Table 1: Hash anchor table

Index	0	1	2	3	4	5	6	7	8	9	10	11
Value	7	3	12	15	13	6	2	4	0	14	8	5

Index	12	13	14	15	16	17	18	19	20	21	22	23
Value	15	11	10	8	5	7	9	10	12	6	13	1

Table 2: Inverted Page Table (IPT)

Index	0	1	2	3	4	5	6	7
PID	0	3	2	3	0	8	0	2
VPN	8	17	8	4	1	3	5	4
Next PTE	2	T	T	4	T	T	10	12

Index	8	9	10	11	12	13	14	15
PID	2	0	8	6	0	2	2	4
VPN	2	3	13	1	2	32	0	4
Next PTE	3	T	0	9	14	8	T	11

Requests to fill in

PID	VPN	TLB	Frame number	TLB hit	Page fault	Latency (ns)
0	1	Hit/Miss		Yes/No	Yes/No	
0	2	Hit/Miss		Yes/No	Yes/No	
0	1	Hit/Miss		Yes/No	Yes/No	
0	3	Hit/Miss		Yes/No	Yes/No	
8	3	Hit/Miss		Yes/No	Yes/No	
0	3	Hit/Miss		Yes/No	Yes/No	
2	2	Hit/Miss		Yes/No	Yes/No	
2	1	Hit/Miss		Yes/No	Yes/No	
8	13	Hit/Miss		Yes/No	Yes/No	
8	3	Hit/Miss		Yes/No	Yes/No	

Solution: The “Index” row of the inverted page table gives the page frame; IPT entries describe the process and VPN within that process. We only count time for page-fault detection (so: time for a TLB check and, on a TLB miss, extra time for the hash anchor and the required number of IPT memory accesses). No hard disk access is needed for detection. A process switch flushes the TLB (therefore the TLB only needs to store the mapping VPN $\rightarrow$ frame number).

f	PID	VPN	TLB (after)	Frame #	TLB hit	Page fault	Latency (ns)
1	0	1	(1,4) -	4	Miss	No	300.5
2	0	2	(1,4) (2,12)	12	Miss	No	200.5
1	0	1	(1,4) (2,12)	4	Hit	No	0.5
3	0	3	(1,4) (3,9)	9	Miss	No	400.5
11	8	3	(3,5) -	5	Miss	No	200.5
3	0	3	(3,9) -	9	Miss	No	400.5
22	2	2	(2,8) -	8	Miss	No	300.5
21	2	1	(2,8) -	/	Miss	Yes	500.5
21	8	13	(13,10) -	10	Miss	No	300.5
11	8	3	(13,10) (3,5)	5	Miss	No	200.5

8 Count the number of page faults

Consider the following page sequence:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

How many page faults occur if this process has 1, 2, ..., 6 or 7 frames available? Do this for the following three replacement algorithms: LRU, FIFO and MIN. Initially, there are no pages in memory.

Solution: MIN replaces the page that will be referenced farthest in the future.

Frames	LRU	FIFO	MIN
1	20	20	20
2	18	18	15
3	15	16	11
4	10	14	8
5	8	10	7
6	7	10	7
7	7	7	7

9 Segmented FIFO

True or false?

Solution idea (diagram):

```
NEW "FIFO"  
L1  
"LRU"  
L2 REPLACED  
HIT
```

1. For all pages in L_1 , they were requested more recently than those in L_2 .

Solution: False.

```
L1: A B  
L2: C D  
=> then requests: B, X =>  
L1: X A  
L2: B C  
Now B was requested more recently than A.
```

2. Pages in L_1 are in LRU order.

Solution: False. They are in FIFO order.

3. Pages in L_2 are in LRU order.

Solution: False. They are in FIFO order.

4. The $|L_2| - 1$ most recently used pages are safe (cannot be replaced).

Solution: True, because they were placed “in front of” L_2 more recently and can therefore survive at least $|L_2|$ replacements.

5. The $|L_2|$ most recently used pages are safe.

Solution: True, for the same reason: they can still survive at least $|L_2|$ replacements.

6. A segmented FIFO system with $|L_1| = |L_2| = 2$ is equivalent to a tree-based pseudo-LRU system with 4 pages.

Solution: False. See caching exercise 4: with tree-based pseudo-LRU, one page can be “protected” by its neighbor; with segmented FIFO, this is not possible.

10 Benchmarking

Draw a graph that illustrates a cache benchmark as described on p. 31 of the theory course. We consider a system with an L1 cache and a TLB cache. The L1 cache size is 256 KByte, a cache line contains 32 bytes, and direct mapping is assumed. An L1 access takes 250ns; an L1 miss adds 500ns; a TLB miss adds another 1000ns. The TLB is 4-way set associative, contains 128 entries, and the page size is 8192 bytes. Make the graph with stride size (log scale) on the x-axis and access time on the y-axis, for array sizes from 4 KByte to 4096 KByte.

Solution (sketch):

```
Time (ns)  
1750 | 2M - 4M (TLB+L1 misses)  
    |
```

750		512K - 1M (L1 misses, TLB hits)	
250		4K - 256K (L1 hits)	
	+	-----	Stride (log)
		2 8 32 128 512 2K 8K 32K 128K 512K 2M	